

Laboratorio: recursión y backtracking

Juan Esteban Ruiz, Esteban Velasquez

MOMENTO 1 — Enunciado del Laboratorio 3

El caso: rutas de recolección en un barrio. Tienen un mapa como matriz, donde cero es calle transitable, uno es calle bloqueada, y hay puntos de acopio marcados con dos. El camión entra por la esquina superior izquierda.

- **Parte A — El camino básico (35%)**

Encuentren un camino desde la entrada hasta la esquina inferior derecha, esquivando los bloqueos, e impriman el mapa con el camino marcado. Esto es el laberinto del lunes, adaptado.

- **Parte B — Recolección obligatoria (40%)**

Ahora el que hace pensar. El camión no puede simplemente llegar a la salida: tiene que pasar por todos los puntos de acopio antes de salir. Encuentren un camino que los recoja todos.

Piénsenlo antes de programar. El estado ya no es solo dónde estoy: es dónde estoy y cuántos puntos llevo recogidos. El caso base cambia: llegar a la salida ya no basta, hay que llegar con todos recogidos.

- **Parte C — Análisis (25%)**

En el archivo de respuestas: cuántos caminos distintos encontró su algoritmo en la Parte A; qué pasa con el tiempo de ejecución cuando el mapa crece de cinco por cinco a diez por diez, medido de verdad, no estimado; y una explicación en cuatro líneas de por qué este problema no se puede resolver bien con ciclos anidados.

Como siempre, los dos lenguajes. Y en C++ compilen con el sanitizer, aunque hoy no haya memoria dinámica: una recursión desbordada también la detecta.

MOMENTO 2 — Desarrollo

- **PARTE A** Si un equipo dice «se cuelga», casi siempre es que no está marcando las casillas visitadas, y da vueltas en círculo. Si dice «no encuentra salida cuando sí hay», es que no está desmarcando.
- **PARTE B** El atasco es de diseño: intentan resolverlo con la misma función de la Parte A. La pregunta que desatasca es «¿qué información necesita llevar la función además de la posición?».
- **PARTE C** Recuérdeles medir de verdad, con la librería de tiempo, no estimar. Es la primera vez que miden y les cuesta.

Antes de arrancar, dos consejos. Uno: empiecen imprimiendo el mapa en cada llamada recursiva, aunque ensucie la salida. Ver por dónde va el algoritmo es la forma más rápida de encontrar el error, y después borran los prints.

Y dos: prueben primero con un mapa de tres por tres que hayan resuelto a mano. Si el algoritmo falla ahí, no van a poder depurarlo en uno de diez por diez.

Momento 1

Con el mapa 5x5 dado, el código de la parte A que es de encontrar caminos, encontró 32 caminos distintos entre (0,0) y la salida (4,4), sin exigir recoger los puntos de acopio, esto lo hizo en un tiempo de \$0.000188s.

Dato aparte: si se exige recoger los 4 puntos de acopio antes de salir que es la Parte B, el conteo sube a 597.816 caminos, y ahí ya se nota el problema de tiempo que se explica abajo.

Usamos `time.perf_counter` y crecimos el mapa de 5X5 a 10X10, y notamos una clara diferencia en el tiempo de ejecución en las dos partes. Si bien la parte A el tiempo se mantiene prácticamente instantáneo, en la parte B el tiempo si incrementa de una forma exponencial pasando de encontrar los caminos en 11.547074s a 170.856059 s

¿Por qué este problema no se puede resolver bien con ciclos anidados?

Con ciclos for anidados necesitarías un nivel de anidamiento por cada paso del camino, pero no sabes de antemano cuántos pasos tiene el camino más largo, así que no hay un número fijo de for que puedas escribir. Además, en cada casilla el recorrido se puede ramificar en hasta 4 direcciones, y si una rama no lleva a ningún lado hay que retroceder y probar otra, además, los ciclos anidados no tienen memoria de "decisiones anteriores" para deshacerlas

Momento 2 — Desarrollo (cómo el código evita los problemas típicos)

Parte A — "se cuelga" / "no encuentra salida cuando sí hay": el código marca `visitado[r][c] = True` antes de explorar los vecinos, y lo vuelve a poner en `False` justo antes de retornar (después del `camino.pop()`). Si no marcara, el algoritmo daría vueltas en círculo entre las mismas casillas (el "se cuelga"). Si marcara pero no desmarcara, una casilla quedaría bloqueada para siempre aunque otro camino sí la necesitara, y ahí es donde un camino que sí existe deja de encontrarse.

Parte B — el atasco de diseño: por eso no reusamos la función de la Parte A. `recoger_puntos` es una función aparte porque el estado ya no es solo la posición (r, c): dos caminos pueden pasar por la misma casilla pero con distintos puntos ya recogidos, y para el problema eso los hace caminos distintos. La respuesta a "¿qué información necesita llevar la función además de la posición?" es la máscara de bits con los puntos de acopio ya visitados, por eso el estado que se guarda en `visitados` es la tupla (r, c, mascara) y no solo (r, c), y por eso `mascara` completa es la condición junto con llegar a la salida para contar un camino como válido.

Parte C — medir, no estimar: se usó `time.perf_counter()` en Python alrededor de la llamada al backtracking, tal como está en el código de arriba. Los números de la tabla son ese tiempo medido directamente, no una proyección hecha a mano.